

The State-Space Recurrence.

A linear state-space model (SSM) is the continuous-time cousin of an RNN: a hidden state $h(t)$ integrates the input $u(t)$ through a linear ODE, and the output $y(t)$ reads the state out. Discretise that ODE with a step Δ and you get a clean linear recurrence $h_t = \bar{A} h_{t-1} + \bar{B} u_t$. The same recurrence has *two faces*: a sequential RNN-like view that costs $O(1)$ per step at inference, and a convolution view $y = u * \bar{K}$ that trains in parallel. We derive both, unroll the recurrence by hand, rebuild the identical outputs from the convolution kernel, and verify they match. Every number below is computed in full — nothing summarised or deferred.

THE EQUATION

$$h'(t) = A h(t) + B u(t), \quad y(t) = C h(t) \quad \xrightarrow{\text{ZOH}, \Delta} \quad h_t = \bar{A} h_{t-1} + \bar{B} u_t, \quad y_t = C h_t$$

Worked example. Diagonal $A = \text{diag}(-1, -2)$, $B = [1, 1]^\top$, $C = [1, 1]$, step $\Delta = 0.5$, inputs $u_{1:4} = [1, 0.5, -1, 2]$. We unroll h_t, y_t for $t = 1 \dots 4$ by the recurrence, then recompute the same y_t from the convolution kernel $\bar{K} = (C\bar{B}, C\bar{A}\bar{B}, \dots)$ and confirm they are identical.

Topic 1 of 2 in this module · companion to the course page · v1.0

Contents

1	The continuous SSM and its discretisation	2
2	The discrete recurrence and its two views	3
3	Worked computation: unrolling the recurrence	4
4	The same output from the convolution kernel	4
5	Why this matters: SSM vs. attention at inference	5
6	Where this sits in the track	5
A	Reproducible (NumPy)	6

1 The continuous SSM and its discretisation

A linear, time-invariant SSM with state $h(t) \in \mathbb{R}^N$, scalar input $u(t) \in \mathbb{R}$ and scalar output $y(t) \in \mathbb{R}$ is the first-order linear ODE

$$h'(t) = A h(t) + B u(t), \quad y(t) = C h(t) + D u(t) \quad (1)$$

with $A \in \mathbb{R}^{N \times N}$, $B \in \mathbb{R}^{N \times 1}$, $C \in \mathbb{R}^{1 \times N}$, $D \in \mathbb{R}$. The $D u$ term is a plain skip connection; we set $D = 0$ below and reinstate it freely at the end. Define every symbol:

symbol	shape	role	here
$h(t)$	$\mathbb{R}^N$	hidden state (memory)	$N = 2$
A	$\mathbb{R}^{N \times N}$	state transition (decay)	$\text{diag}(-1, -2)$
B	$\mathbb{R}^{N \times 1}$	input→state gain	$[1, 1]^\top$
C	$\mathbb{R}^{1 \times N}$	state→output readout	$[1, 1]$
Δ	$\mathbb{R}_{>0}$	discretisation step	0.5

To run this on a discrete sequence $u_1, u_2, \dots$ we sample at spacing Δ and hold u constant on each interval — the **zero-order hold** (ZOH). Solving the ODE exactly over one interval of length Δ with constant input gives

$$\bar{A} = \exp(\Delta A), \quad \bar{B} = A^{-1}(\exp(\Delta A) - I) B \quad (2)$$

Derivation. The ODE's solution over $[t, t + \Delta]$ with u held at u_{t+1} is $h(t + \Delta) = e^{\Delta A}h(t) + (\int_0^\Delta e^{\tau A} d\tau)B u_{t+1}$. The integral evaluates to $\int_0^\Delta e^{\tau A} d\tau = A^{-1}(e^{\Delta A} - I)$ (differentiate the right side to check), giving (2).

A says how fast the state forgets: a continuous decay rate. Discretising turns the rate into a per-step *multiplier* $\bar{A} = e^{\Delta A}$ (always between 0 and 1 for negative real A), and $\bar{B}$ is the exact amount of the held input that gets integrated into the state over one step. ZOH is not an approximation of the recurrence — it is the *exact* state at the sample points.

Diagonal A . If $A = \text{diag}(\lambda_1, \dots, \lambda_N)$ then $\exp(\Delta A)$ and A^{-1} act elementwise, so (2) becomes, per coordinate i ,

$$\bar{A}_i = e^{\Delta \lambda_i}, \quad \bar{B}_i = \frac{e^{\Delta \lambda_i} - 1}{\lambda_i} B_i.$$

With $\lambda = (-1, -2)$, $\Delta = 0.5$, $B = (1, 1)$:

$$\bar{A} = (e^{-0.5}, e^{-1}) = (0.606531, 0.367879), \quad \bar{B} = \left(\frac{e^{-0.5} - 1}{-1}, \frac{e^{-1} - 1}{-2} \right) = (0.393469, 0.316060).$$

2 The discrete recurrence and its two views

With the discrete matrices in hand, the SSM *is* a linear recurrence:

$$\boxed{h_t = \bar{A} h_{t-1} + \bar{B} u_t, \quad y_t = C h_t} \quad h_0 = 0. \quad (3)$$

2.1 View (i): recurrent — $O(1)$ per step

Equation (3) is read literally: keep one state vector $h \in \mathbb{R}^N$, and at each token do one state-update and one readout. Per step that is $O(N)$ work (here $N=2$, a constant) and $O(N)$ memory; over T tokens, $O(TN)$ time and a *fixed* $O(N)$ memory — nothing grows with T . This is exactly an RNN, and it is ideal for autoregressive **inference**: emit a token, fold it into h , repeat.

2.2 View (ii): convolutional — parallel over t

Unroll (3) from $h_0 = 0$. Substituting repeatedly,

$$h_t = \sum_{k=0}^{t-1} \bar{A}^k \bar{B} u_{t-k}, \quad y_t = C h_t = \sum_{k=0}^{t-1} \underbrace{C \bar{A}^k \bar{B}}_{=\bar{K}_k} u_{t-k}.$$

So y is the **causal convolution** of u with a fixed kernel $\bar{K}$:

$$\boxed{y = u * \bar{K}, \quad \bar{K} = (C \bar{B}, C \bar{A} \bar{B}, C \bar{A}^2 \bar{B}, \dots)} \quad (4)$$

The kernel does not depend on the data, so for **training** we precompute $\bar{K}$ once and apply the whole length- T convolution in parallel (an FFT makes it $O(T \log T)$). Same y as view (i), computed without a sequential loop.

One model, two algorithms. The *recurrent* form (3) is $O(1)$ -state, sequential — best for generation. The *convolutional* form (4) is parallel over time — best for training. Because the recurrence is *linear*, the two are algebraically identical: training and inference use the *same* weights, just unrolled differently.

3 Worked computation: unrolling the recurrence

Inputs $u_{1:4} = [1, 0.5, -1, 2]$, $\bar{A} = (0.606531, 0.367879)$, $\bar{B} = (0.393469, 0.316060)$, $C = [1, 1]$, $h_0 = (0, 0)$. Apply (3) coordinatewise; $y_t = h_t^{(1)} + h_t^{(2)}$.

$t = 1$ ($u_1 = 1$): $h_1 = \bar{A} \cdot 0 + \bar{B} \cdot 1 = (0.393469, 0.316060)$, $y_1 = 0.709530$.

$t = 2$ ($u_2 = 0.5$): $h_2^{(1)} = 0.606531(0.393469) + 0.393469(0.5) = 0.435386$; $h_2^{(2)} = 0.367879(0.316060) + 0.316060(0.5) = 0.274302$; $y_2 = 0.709688$.

$t = 3$ ($u_3 = -1$): $h_3^{(1)} = 0.606531(0.435386) - 0.393469 = -0.129394$; $h_3^{(2)} = 0.367879(0.274302) - 0.316060 = -0.215150$; $y_3 = -0.344545$.

$t = 4$ ($u_4 = 2$): $h_4^{(1)} = 0.606531(-0.129394) + 0.393469(2) = 0.708457$; $h_4^{(2)} = 0.367879(-0.215150) + 0.316060(2) = 0.552971$; $y_4 = 1.261428$.

t	u_t	$h_t^{(1)}$	$h_t^{(2)}$	$y_t = Ch_t$
1	1.0	0.393469	0.316060	0.709530
2	0.5	0.435386	0.274302	0.709688
3	-1.0	-0.129394	-0.215150	-0.344545
4	2.0	0.708457	0.552971	1.261428

4 The same output from the convolution kernel

Build $\bar{K}_k = C\bar{A}^k\bar{B} = \bar{A}_1^k\bar{B}_1 + \bar{A}_2^k\bar{B}_2$ for $k = 0, 1, 2, 3$:

$$\bar{K}_0 = 0.393469 + 0.316060 = 0.709530,$$

$$\bar{K}_1 = 0.606531(0.393469) + 0.367879(0.316060) = 0.354923,$$

$$\bar{K}_2 = 0.606531^2(0.393469) + 0.367879^2(0.316060) = 0.187523,$$

$$\bar{K}_3 = 0.606531^3(0.393469) + 0.367879^3(0.316060) = 0.103531.$$

The kernel decays geometrically — a coloured strip (intensity $\propto$ magnitude, capped at **indigo!80**):

	$\bar{K}_0$	$\bar{K}_1$	$\bar{K}_2$	$\bar{K}_3$
kernel tap	0.709530	0.354923	0.187523	0.103531

Now $y_t = \sum_{k=0}^{t-1} \bar{K}_k u_{t-k}$:

$$y_1 = \bar{K}_0 u_1 = 0.709530(1) = 0.709530,$$

$$y_2 = \bar{K}_0 u_2 + \bar{K}_1 u_1 = 0.709530(0.5) + 0.354923(1) = 0.709688,$$

$$y_3 = \bar{K}_0 u_3 + \bar{K}_1 u_2 + \bar{K}_2 u_1 = 0.709530(-1) + 0.354923(0.5) + 0.187523(1) = -0.344545,$$

$$y_4 = \bar{K}_0 u_4 + \bar{K}_1 u_3 + \bar{K}_2 u_2 + \bar{K}_3 u_1 \\ = 0.709530(2) + 0.354923(-1) + 0.187523(0.5) + 0.103531(1) = 1.261428.$$

They match the recurrence exactly — maximum absolute difference $\approx 5.6 \times 10^{-17}$ (floating-point round-off). The two views are the same function.

The convolution kernel *is* the model’s impulse response: $\bar{K}_k$ is how strongly an input k steps ago still influences the output now. A decaying $\bar{K}$ ($|\bar{A}| < 1$) means the model remembers the recent past most. Choosing A (Topic 2’s HiPPO) is precisely choosing the *shape* of this memory kernel.

5 Why this matters: SSM vs. attention at inference

For autoregressive decoding the SSM is dramatically lighter than attention, because its memory is the fixed-size state $h \in \mathbb{R}^N$ rather than a growing cache of all past keys/values:

	SSM (recurrent)	Attention (KV-cache)
time per new token	$O(N)$ (constant)	$O(T)$ (grows with context)
total time, T tokens	$O(TN)$	$O(T^2)$
state / cache memory	$O(N)$ <i>fixed</i>	$O(Td)$ <i>grows with T</i>
training	parallel via conv. (4)	parallel via $QK^\top$

At long context the difference is decisive: attention’s cost and memory grow with T , while the SSM state stays the same size whether it has seen 10 or 10,000 tokens.

That fixed-size state is also the catch. Attention can re-read any earlier token *exactly* — a perfect lookup table over the whole context. The SSM must compress all of history into the N numbers of h_t , so it cannot losslessly recall arbitrary past tokens (e.g. “copy the 5000th token verbatim”). A bounded state is an information bottleneck; this is the core accuracy-vs-efficiency trade against attention, and it is exactly what Mamba’s *selectivity* (Topic 2) tries to soften.

6 Where this sits in the track

This opens Module 7.2. The recurrence (3) and its convolution twin (4) are the skeleton of every modern SSM. Topic 2 makes A *diagonal* (so the N scalar recurrences run independently), shows the recurrence is an associative *scan* computable in $O(\log T)$ depth, initialises A with **HiPPO** for long-range memory, and notes how **Mamba** makes A, B, C, Δ input-dependent — which breaks the convolution view (4) and forces the parallel scan instead.

A Reproducible (NumPy)

Inputs: $A = \text{diag}(-1, -2)$, $B = C^T = [1, 1]$, $\Delta = 0.5$, $u = [1, 0.5, -1, 2]$. The snippet builds $\bar{A}$, $\bar{B}$ by ZOH, unrolls the recurrence, builds the convolution kernel, and checks they agree.

```
import numpy as np
A=np.array([-1.,-2.]); B=np.array([1.,1.]); C=np.array([1.,1.])
Delta=0.5; u=np.array([1.,0.5,-1.,2.])
Abar=np.exp(Delta*A)                # [0.606531 0.367879]
Bbar=(np.exp(Delta*A)-1.)/A*B       # [0.393469 0.316060]
h=np.zeros(2); ys=[]
for ut in u:
    h=Abar*h+Bbar*ut; ys.append(C@h) # recurrence
K=[C@((Abar**k)*Bbar) for k in range(4)] # [0.70953 0.354923 0.187523 0.103531]
yc=[sum(K[k]*u[t-k] for k in range(t+1)) for t in range(4)]
print(np.round(ys,6)) # [ 0.70953 0.709688 -0.344545 1.261428]
print(np.round(yc,6)) # identical
print(np.max(np.abs(np.array(ys)-np.array(yc)))) # ~5.6e-17
```